PEMBELAJARAN MESIN (MACHINE LEARNING)

MINGGU 1

# Pengenalan Machine Learning & Ekosistem ML Modern

*Sub-CPMK8.1.1 • Fondasi Pemahaman ML*

# Capaian Pembelajaran Minggu Ini

**CLO1** Memahami definisi Machine Learning dan perbedaannya dengan pemrograman tradisional

**CLO2** Menjelaskan taksonomi algoritma ML: Supervised, Unsupervised, dan Reinforcement Learning

**CLO3** Memahami ML Pipeline dari data collection hingga model deployment

**CLO4** Mengenal ekosistem tools ML modern: scikit-learn, PyTorch, HuggingFace, MLflow

**CLO5** Membedakan ML Klasik, Deep Learning, dan AI Generatif

# Agenda Pertemuan

1. Apa itu Machine Learning? Definisi & Motivasi
2. Sejarah & Evolusi ML: Dari Perceptron ke GPT
3. Taksonomi Algoritma ML: Supervised, Unsupervised, RL
4. ML Pipeline: Data → Features → Model → Evaluation → Deploy
5. Ekosistem Tools ML Modern (2026)
6. ML vs DL vs AI Generatif: Perbandingan
7. Studi Kasus & Demo Singkat
8. Tugas-1 & Referensi

1

# Apa itu Machine Learning?

Definisi, Motivasi, dan Mengapa ML Penting di 2026

# Definisi Machine Learning

- Arthur Samuel (1959): "Field of study that gives computers the ability to learn without being explicitly programmed"
- Tom Mitchell (1997): "A computer program is said to learn from experience E with respect to some task T and performance measure P, if its performance at T, as measured by P, improves with experience E"
- Definisi Modern (2026): ML adalah cabang AI yang memungkinkan sistem belajar pola dari data untuk membuat prediksi atau keputusan tanpa diprogram secara eksplisit untuk setiap skenario
- Perbedaan kunci: Traditional Programming (Rules + Data → Output) vs ML (Data + Output → Rules/Model)

# Traditional Programming vs Machine Learning

## Traditional Programming

- Input: Aturan/rules yang ditulis manual
- Proses: Rule-based logic (if-else)
- Semakin kompleks masalah, semakin banyak rules
- Sulit menangani data unstructured
- Contoh: Spam filter dengan keyword list

## Machine Learning

- Input: Data contoh (training data)
- Proses: Algoritma belajar pola dari data
- Model dapat menangani kompleksitas tinggi
- Excellent untuk pattern recognition
- Contoh: Spam classifier berbasis probabilitas

# Mengapa Machine Learning Penting di 2026?

**$503B**

**Market Size 2030**

Proyeksi pasar ML global (Grand View Research, 2025)

**150%**

**Job Growth YoY**

Peningkatan lowongan ML engineer 2024-2025

**71%**

**Python Adoption**

Job posting ML yang membutuhkan Python

**$213K**

**Avg. Salary (US)**

Rata-rata gaji ML Engineer senior di Amerika Serikat

# Aplikasi ML di Berbagai Industri

- Healthcare: Diagnosis penyakit dari medical imaging (CNN), drug discovery, prediksi epidemi
- Finance: Fraud detection, credit scoring, algorithmic trading, risk assessment
- E-commerce: Recommendation system (Netflix, Spotify), dynamic pricing, customer segmentation
- NLP & LLM: ChatGPT, machine translation, sentiment analysis, document summarization
- Computer Vision: Self-driving cars (Tesla), facial recognition, quality control manufaktur
- Manufacturing: Predictive maintenance, anomaly detection, quality control
- Agriculture: Crop disease detection, yield prediction, precision farming

2

# Sejarah & Evolusi ML

Dari Perceptron (1958) hingga GPT-4 dan Beyond (2026)

# Timeline Evolusi Machine Learning

- 1943: McCulloch-Pitts – Model neuron matematika pertama
- 1957-58: Frank Rosenblatt – Perceptron, neural network pertama
- 1967: Nearest Neighbor algorithm (Cover & Hart)
- 1980s: Backpropagation (Rumelhart, Hinton, Williams, 1986) – Neural network training
- 1995: Support Vector Machine (Vapnik & Cortes) – Kernel methods
- 1997: LSTM (Hochreiter & Schmidhuber) – Sequence modeling breakthrough
- 2001: Random Forest (Breiman) – Ensemble methods
- 2006: Deep Learning revolution dimulai (Hinton et al.) – Deep belief networks

# Timeline Evolusi ML (lanjutan): Era Modern

- 2012: AlexNet memenangkan ImageNet – CNN revolution, GPU computing
- 2014: GAN (Goodfellow et al.) – Generative Adversarial Networks
- 2014: Adam Optimizer (Kingma & Ba) – Standard DL optimizer
- 2017: "Attention Is All You Need" (Vaswani et al.) – Transformer architecture → mengubah segalanya
- 2018: BERT (Devlin et al.) – Bidirectional pre-training untuk NLP
- 2020: GPT-3 (Brown et al.) – 175B parameters, few-shot learning
- 2022: ChatGPT – Mendunia, RLHF, AI untuk semua orang
- 2023-2026: GPT-4, LLaMA, Mistral, Gemini, Claude – Era AI Generatif, Multimodal, Agentic AI

# Tiga Era Machine Learning

## Era 1: ML Klasik (1950s-2010s)

- Algoritma: Linear/Logistic Regression, SVM, Decision Trees, Random Forest, k-NN
- Data: Structured/tabular data (spreadsheet)
- Feature engineering manual oleh domain expert
- Tools: scikit-learn, R, Weka
- Masih sangat relevan untuk data tabular!

## Era 2: Deep Learning (2012-now)

- Algoritma: CNN, RNN/LSTM, GAN, Autoencoder
- Data: Unstructured (gambar, teks, audio, video)
- Feature learning otomatis dari data mentah
- Tools: PyTorch, TensorFlow
- GPU/TPU computing menjadi kunci

# Era 3: AI Generatif & Foundation Models (2020-sekarang)

- Transformer architecture menjadi backbone semua model state-of-the-art
- Pre-training pada massive dataset (internet-scale) → Fine-tuning pada task spesifik
- Large Language Models (LLM): GPT-4, Claude, Gemini, LLaMA, Mistral
- Multimodal AI: Teks + Gambar + Audio + Video dalam satu model
- Emerging: Agentic AI – AI yang bisa menggunakan tools, merencanakan, dan berkolaborasi
- RLHF (Reinforcement Learning from Human Feedback) untuk alignment
- Parameter-Efficient Fine-Tuning: LoRA, QLoRA, DoRA → Fine-tune LLM dengan resource minimal
- RAG (Retrieval-Augmented Generation): Menggabungkan retrieval + generation

3

# Taksonomi Algoritma ML

Supervised, Unsupervised, dan Reinforcement Learning

# Tiga Paradigma Utama Machine Learning

- Supervised Learning: Belajar dari data berlabel (input-output pairs). Goal: memprediksi output untuk input baru
- Unsupervised Learning: Belajar dari data tanpa label. Goal: menemukan pola/struktur tersembunyi dalam data
- Reinforcement Learning: Agent belajar melalui interaksi dengan environment. Goal: memaksimalkan cumulative reward
- Semi-supervised Learning: Kombinasi sedikit data berlabel + banyak data tidak berlabel
- Self-supervised Learning: Model membuat label sendiri dari data (pretext task) – basis pre-training LLM

# Supervised Learning: Classification vs Regression

## Classification (Kategorikal)

- Output: Label diskrit/kategori
- Binary: Spam/Not Spam, Sick/Healthy
- Multi-class: Cat/Dog/Bird, Digit 0-9
- Algoritma: Logistic Regression, SVM, Decision Tree, Random Forest, Neural Network
- Metrik: Accuracy, Precision, Recall, F1-Score, AUC-ROC

## Regression (Numerik)

- Output: Nilai kontinu/numerik
- Contoh: Harga rumah, suhu besok, stock price
- Algoritma: Linear Regression, SVR, Decision Tree Regressor, Neural Network
- Metrik: MSE, RMSE, MAE, R-squared
- Overfitting: Model terlalu fit ke training data

# Unsupervised Learning: Clustering & Dimensionality Reduction

- Clustering: Mengelompokkan data yang mirip tanpa label
- → K-Means, DBSCAN, Hierarchical Clustering, Gaussian Mixture Models
- → Aplikasi: Customer segmentation, anomaly detection, image segmentation
- Dimensionality Reduction: Mengurangi jumlah fitur sambil mempertahankan informasi
- → PCA (Principal Component Analysis), t-SNE, UMAP
- → Aplikasi: Visualization, noise reduction, feature compression
- Association Rules: Menemukan aturan asosiasi dalam data transaksional
- → Apriori, FP-Growth – Market basket analysis
- Generative Models: GAN, VAE – Membuat data baru yang mirip dengan training data

# Reinforcement Learning (RL)

- Agent berinteraksi dengan Environment melalui Actions untuk mendapat Rewards
- Goal: Menemukan optimal policy ($\pi^*$) yang memaksimalkan expected cumulative reward
- Key concepts: State (s), Action (a), Reward (r), Policy ($\pi$), Value Function (V)
- Exploration vs Exploitation trade-off: Mencoba aksi baru vs menggunakan aksi terbaik yang diketahui
- Algoritma Klasik: Q-Learning, SARSA, Policy Gradient
- Deep RL: DQN (Atari), A3C, PPO (Proximal Policy Optimization)
- Aplikasi: Game AI (AlphaGo, AlphaStar), Robotics, Autonomous Driving
- RLHF: RL dari feedback manusia – kunci di balik ChatGPT dan Claude

4

# ML Pipeline

Dari Raw Data hingga Model Deployment

# ML Pipeline: 7 Tahapan Utama

- 1. Problem Definition: Definisikan masalah bisnis dan terjemahkan ke masalah ML
- 2. Data Collection: Kumpulkan data dari berbagai sumber (database, API, web scraping, sensors)
- 3. Data Preprocessing & EDA: Cleaning, handling missing values, outliers, exploratory analysis
- 4. Feature Engineering: Transformasi data mentah menjadi fitur yang informatif untuk model
- 5. Model Training & Selection: Pilih dan latih algoritma, cross-validation, hyperparameter tuning
- 6. Model Evaluation: Evaluasi performa pada test set, metrik sesuai task (accuracy, F1, RMSE)
- 7. Deployment & Monitoring: Deploy model ke production, monitor performa, retrain jika perlu

# Data Preprocessing: Langkah Kritis

- Missing Values: Imputation (mean, median, mode, KNN), atau drop rows/columns
- Outlier Detection: Z-score, IQR method, Isolation Forest
- Feature Scaling: StandardScaler (z-score), MinMaxScaler (0-1), RobustScaler
- Encoding Categorical: One-Hot Encoding, Label Encoding, Target Encoding
- Text Preprocessing: Tokenization, lowercasing, stopword removal, stemming/lemmatization
- Image Preprocessing: Resize, normalize (0-1 atau ImageNet mean/std), augmentation
- Train/Validation/Test Split: Typically 70/15/15 atau 80/10/10
- Stratified Split: Penting untuk imbalanced dataset agar distribusi kelas terjaga

# Feature Engineering: Seni Membuat Fitur yang Baik

- "Feature engineering is the key to successful machine learning" – Andrew Ng
- Numerical: Polynomial features, log transform, binning, interaction features
- Temporal: Ekstrak year, month, day_of_week, is_weekend dari datetime
- Text: TF-IDF, word embeddings (Word2Vec, GloVe), sentence embeddings
- Domain Knowledge: Fitur spesifik industri (medical: BMI dari height+weight)
- Automated: FeatureTools, AutoFeat – otomatis generate fitur baru
- Feature Selection: Filter (correlation), Wrapper (RFE), Embedded (Lasso, tree importance)
- Modern: Deep learning melakukan feature learning otomatis dari raw data!

# Contoh: ML Pipeline Sederhana dengan scikit-learn

Python

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# 1. Load data
df = pd.read_csv("data.csv")

# 2. Preprocessing
X = df.drop("target", axis=1)
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
```

- Pipeline dasar: load → split → scale → train → evaluate
- StandardScaler: fit pada train, transform pada train & test (hindari data leakage!)
- stratify=y: Menjaga distribusi kelas tetap seimbang di train & test

5

# Ekosistem Tools ML Modern

Python Libraries & Frameworks 2026

# Python ML Ecosystem 2026

**87K⭐**

**PyTorch**

Framework DL dominan untuk riset dan produksi

**145K⭐**

**HuggingFace**

1M+ model, platform AI terbesar

**62K⭐**

**scikit-learn**

Library ML klasik, GPU support baru

**126K⭐**

**LangChain**

Framework untuk aplikasi LLM

# ML Klasik vs Deep Learning Tools

## Classical ML (scikit-learn)

- scikit-learn 1.8+: GPU via Array API, free-threaded Python
- XGBoost, LightGBM, CatBoost: Gradient boosting state-of-the-art
- Pandas 2.x: Data manipulation & analysis
- Optuna: Hyperparameter optimization (Bayesian)
- SHAP: Model explainability

## Deep Learning (PyTorch ecosystem)

- PyTorch 2.10: torch.compile, quantization, LLM support
- HuggingFace Transformers 5.x: 400+ model architectures
- timm: PyTorch Image Models (ViT, ResNet, etc.)
- Lightning: High-level PyTorch training framework
- torchvision, torchaudio: Domain-specific utilities

# LLM & MLOps Tools

## LLM & GenAI Tools

- HuggingFace Hub: 1M+ models, datasets, spaces
- LangChain / LlamaIndex: LLM app frameworks
- PEFT library: LoRA, QLoRA, AdaLoRA fine-tuning
- vLLM / SGLang: High-performance LLM inference
- Unsloth: Fast & memory-efficient fine-tuning

## MLOps & Deployment

- MLflow 3.x: Experiment tracking, model registry, GenAI support
- Weights & Biases: Experiment tracking & visualization
- Docker: Containerization untuk reproducibility
- FastAPI / Gradio: Model serving & demo apps
- DVC: Data version control

# Visualization & EDA Tools

- Matplotlib: Library plotting dasar Python, foundation untuk semua visualization
- Seaborn: Statistical visualization built on Matplotlib, lebih aesthetic
- Plotly: Interactive charts, ideal untuk dashboard dan Jupyter notebooks
- Streamlit: Rapid prototyping web apps untuk ML demos
- Gradio: Quick ML demos dengan interface web, integrasi HuggingFace Spaces
- Jupyter Notebook / JupyterLab: Interactive development environment standar ML
- Google Colab: Free cloud Jupyter dengan GPU (T4/A100) gratis
- VS Code + Python Extensions: IDE modern untuk development ML profesional

6

# ML vs DL vs AI Generatif

Perbandingan dan Kapan Menggunakan Masing-masing

# Perbandingan: ML Klasik vs Deep Learning vs AI Generatif

- ML Klasik: Data tabular (<10K fitur), interpretable, cepat training, perlu feature engineering
- → Best for: Prediksi harga, fraud detection, customer churn, A/B testing
- Deep Learning: Data unstructured (gambar, teks, audio), auto feature learning, butuh GPU
- → Best for: Image classification, object detection, speech recognition, NLP tasks
- AI Generatif: Pre-trained foundation models, few-shot/zero-shot capability, massive scale
- → Best for: Text generation, code generation, image creation, chatbots, RAG systems
- Key insight: XGBoost MASIH mengalahkan DL untuk banyak task data tabular! (Shwartz-Ziv, 2022)
- Pemilihan algoritma harus berdasarkan: jenis data, ukuran dataset, kebutuhan interpretability, resources

# Kapan Menggunakan Apa?

| | | |
|---|---|---|
| 1 | **Data Tabular + <100K rows** | → Logistic Regression, Random Forest, XGBoost |
| 2 | **Data Tabular + >100K rows** | → XGBoost/LightGBM, atau Deep Learning (TabNet) |
| 3 | **Image/Video Data** | → CNN (ResNet, EfficientNet) atau ViT + Transfer Learning |
| 4 | **Text/NLP Tasks** | → BERT fine-tuning, atau LLM (GPT, LLaMA) + PEFT |
| 5 | **Generative Tasks** | → LLM (GPT, Claude), Diffusion Models (Stable Diffusion) |

7

# Studi Kasus & Demo

ML dalam Praktik Nyata

# Studi Kasus: Rekomendasi Film (Netflix/Spotify)

- Problem: Bagaimana merekomendasikan konten yang relevan untuk jutaan pengguna?
- Data: User ratings, watch history, demographics, item features (genre, actors)
- Pendekatan 1 - Collaborative Filtering: Rekomendasi berdasarkan user serupa
- → Matrix Factorization (SVD), k-NN pada user-item matrix
- Pendekatan 2 - Content-Based: Rekomendasi berdasarkan kesamaan item
- → TF-IDF pada deskripsi, embedding similarity
- Pendekatan 3 - Hybrid + Deep Learning: Neural Collaborative Filtering, two-tower model
- Evaluasi: NDCG, MAP, Hit Rate, A/B testing pada production traffic
- Scale: Netflix menjalankan model ini untuk 240M+ subscribers secara real-time!

# Studi Kasus: Fraud Detection (Fintech)

- Problem: Mendeteksi transaksi fraud dari jutaan transaksi per hari
- Challenge: Extreme class imbalance (≤0.1% fraud), real-time inference, evolving patterns
- Data: Transaction amount, merchant, time, location, device, user history
- Solusi ML: Ensemble (XGBoost + LightGBM) dengan SMOTE/ADASYN untuk imbalance
- Feature Engineering: Velocity features (transactions/hour), deviation from normal pattern
- Metrik Kritis: Precision (minimize false positives), Recall (catch fraud), F1-Score
- Deployment: Real-time scoring via FastAPI, <100ms latency requirement
- MLOps: Continuous monitoring, model retraining mingguan karena concept drift

# Demo: Quick ML Pipeline (Google Colab Ready)

Python

```python
# Jalankan di Google Colab!
!pip install scikit-learn pandas matplotlib seaborn

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

# Load dataset
```

- Dataset Iris: 150 sampel, 4 fitur, 3 kelas bunga
- Random Forest: Ensemble dari 100 decision trees
- Akurasi ~97.8% dengan pipeline sederhana!

# Referensi & Sumber


- Bishop, C.M. & Bishop, H. (2024). Deep Learning: Foundations and Concepts. Springer. ISBN: 978-3-031-45467-7.
- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 3rd ed. O'Reilly.
- Prince, S.J.D. (2023). Understanding Deep Learning. MIT Press. ISBN: 978-0-262-04864-4. [Free: udlbook.github.io]
- Murphy, K.P. (2022). Probabilistic Machine Learning: An Introduction. MIT Press. [Free: probml.github.io]
- Zhang, A. et al. (2023). Dive into Deep Learning. d2l.ai. [Free: d2l.ai]
- Raschka, S. (2022). Machine Learning with PyTorch and Scikit-Learn. Packt.
- Stanford CS229 (2025): cs229.stanford.edu | MIT 6.390: ocw.mit.edu | Berkeley CS189: eecs189.org
- ACM/IEEE/AAAI CS2023 Computing Curricula: csed.acm.org/cs2023
- Shwartz-Ziv, R. & Armon, A. (2022). Tabular Data: Deep Learning is Not All You Need. Information Fusion.

# Rangkuman Minggu 1

- Machine Learning: Sistem yang belajar pola dari data tanpa diprogram eksplisit untuk setiap skenario
- Tiga paradigma utama: Supervised (labels), Unsupervised (no labels), Reinforcement (rewards)
- ML Pipeline: Problem → Data → Preprocessing → Feature Eng. → Model → Evaluate → Deploy
- Ekosistem 2026: scikit-learn (klasik), PyTorch (DL), HuggingFace (LLM), MLflow (MLOps)
- ML Klasik masih sangat relevan untuk data tabular! XGBoost ≥ Deep Learning di banyak task tabular
- AI Generatif (LLM, Diffusion) membuka era baru: few-shot learning, agents, multimodal AI
- Minggu depan: Linear Regression & Gradient Descent – fondasi matematika ML!

# Terima Kasih

Minggu 1: Pengenalan Machine Learning
& Ekosistem ML Modern

*Pertanyaan & Diskusi*